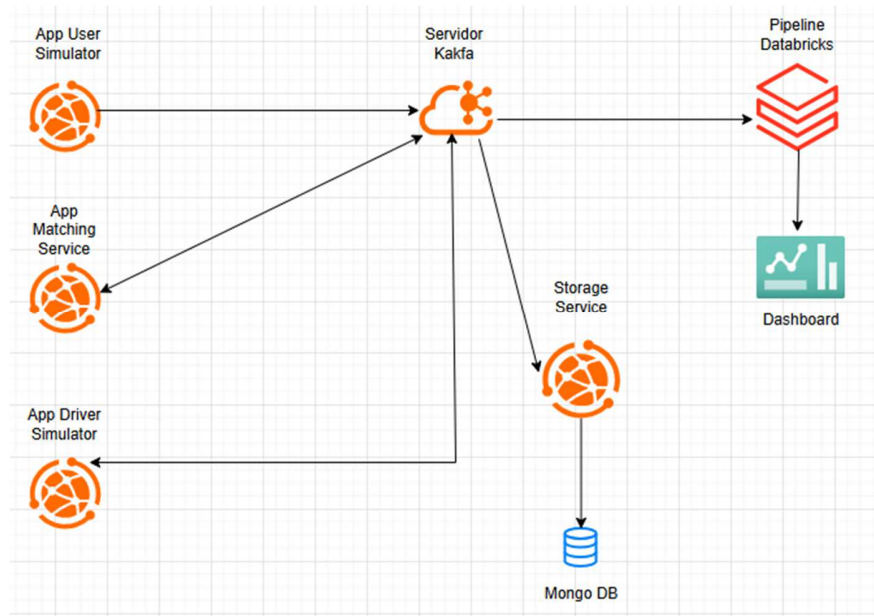


Event-Driven Architecture – Uber Simulation

Project Overview

Diagrama:



Este proyecto implementa una **arquitectura orientada a eventos (Event-Driven Architecture – EDA)** simulando el funcionamiento de una aplicación tipo Uber.

El objetivo es construir un sistema distribuido donde múltiples componentes interactúan a través de eventos en tiempo real, utilizando Apache Kafka como broker de mensajería.

Objetivo

Simular un ecosistema donde:

- 🧑 Usuarios solicitan viajes
 - 🚗 Conductores aceptan solicitudes
 - ⚡ Se generan y procesan eventos en tiempo real
 - 📡 Los datos fluyen a través de Kafka
 - 💾 La información se almacena para análisis histórico
-

Arquitectura del Sistema

Ride Request (User)



Kafka Topic: ride_requests



Driver Service



Kafka Topic: ride_assignments



Trip Updates



Kafka Topic: ride_events



MongoDB (Histórico)



Databricks (Análisis y Dashboard)

Componentes Principales

Apache Kafka

- Actúa como **broker de eventos**
 - Gestiona la comunicación asíncrona entre servicios
 - Se utilizan 3 topics principales:
 - ride_requests
 - ride_assignments
 - ride_events
-

User Simulator

- Simula usuarios solicitando viajes
- Publica eventos en el topic:

- ride_requests

Driver Simulator

- Consume eventos desde ride_requests
- Procesa solicitudes de viaje
- Publica asignaciones en:
 - ride_assignments

Matching Service

- Consume eventos desde ride_assignments
- Gestiona el ciclo de vida del viaje:
 1. Genera evento de **inicio de viaje**
 2. Simula duración del viaje
 3. Genera evento de **finalización**
- Publica eventos en:
 - ride_events

Storage Service (MongoDB)

- Consume todos los eventos generados
- Almacena información en colecciones
- Permite construir un **histórico de viajes**

Databricks (Streaming & Analytics)

- Consume eventos en streaming desde Kafka
- Realiza transformación de datos
- Almacena resultados en tablas analíticas
- Construye dashboards en tiempo real

Flujo de Funcionamiento

1. El **User Simulator** genera una solicitud de viaje
 2. El evento se publica en **Kafka (ride_requests)**
 3. El **Driver Simulator** consume y asigna un conductor
 4. Se genera un evento en **ride_assignments**
 5. El **Matching Service** inicia y finaliza el viaje
 6. Se generan eventos en **ride_events**
 7. Todos los eventos se almacenan en **MongoDB**
 8. **Databricks** consume datos en streaming y genera analítica
-

Conceptos Aplicados

- Event-Driven Architecture (EDA)
 - Streaming de datos en tiempo real
 - Procesamiento asíncrono
 - Sistemas desacoplados
 - Persistencia de eventos (Event Storage)
 - Analítica en tiempo real
-

Beneficios de la Arquitectura

-  Alta escalabilidad
 -  Bajo acoplamiento entre servicios
 -  Procesamiento en tiempo real
 -  Tolerancia a fallos
 -  Capacidad de análisis histórico y streaming
-

Posibles Mejoras

-  Implementar seguridad en Kafka (SSL/SASL)
-  Despliegue en entornos cloud (Azure / AWS)
-  Optimización de dashboards en Databricks

-  Integración de Machine Learning (predicción de demanda)
-  Monitoreo y logging (Prometheus / Grafana)

Autor

Cristian Alvarado

Data Engineer | Databricks Certified | Future Data Architect

-  LinkedIn:
<https://www.linkedin.com/in/cristian-alvarado-cruz-5500a4117/>
-  Email:
c_alvarado_cruz@hotmail.com